

DOSSIER PROJET

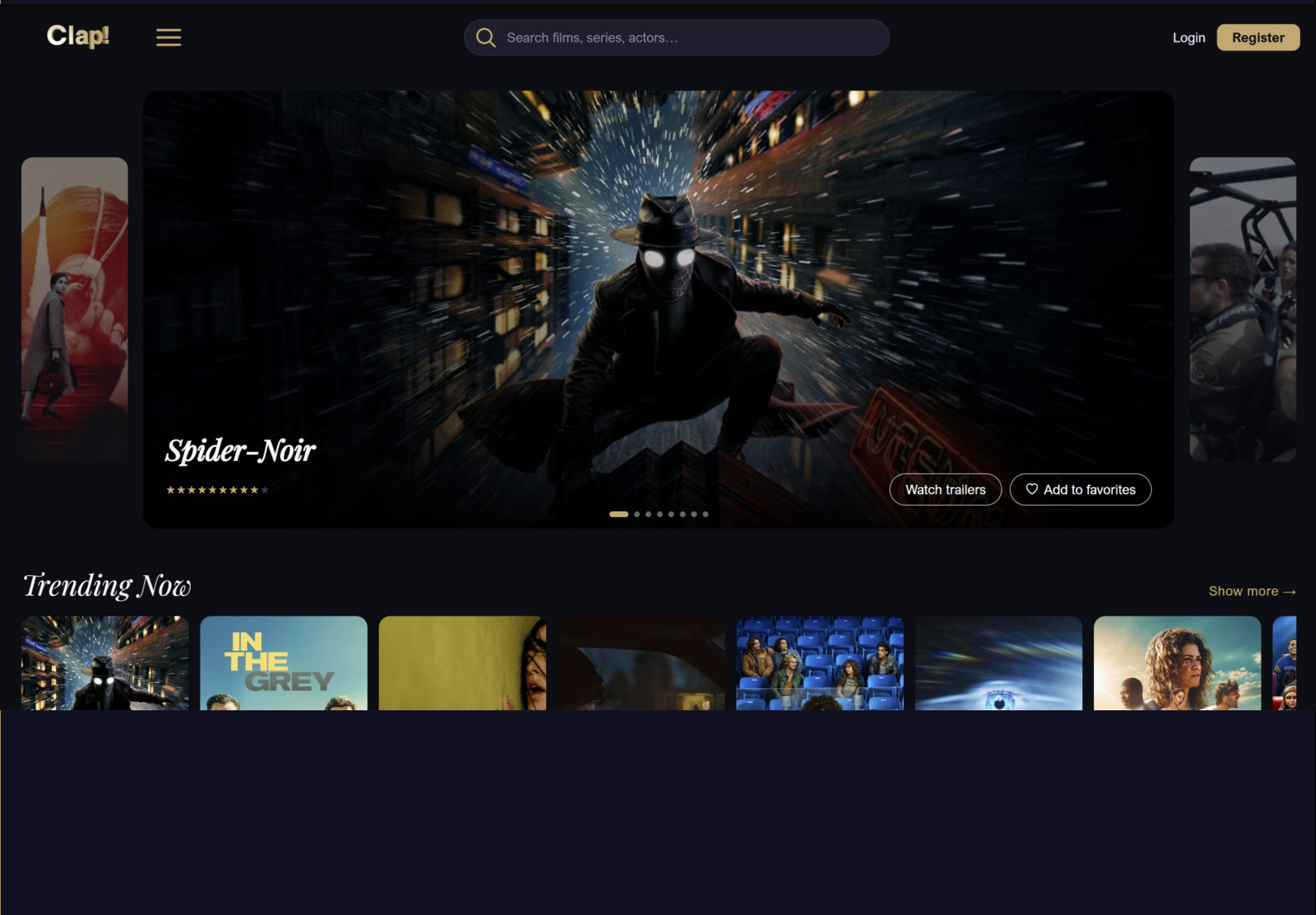
Titre Professionnel — Développeur Web et Web Mobile

Antonin Tacchi · DWWM · 2025 – 2026

CLAP!

Plateforme de découverte de films et séries

Application web full-stack · Architecture microservices · Java Spring Boot · React 18



S O M M A I R E

01	INTRODUCTION	p. 3
02	COMPÉTENCES MISES EN ŒUVRE	p. 4
03	RÉSUMÉ DU PROJET	p. 6
04	CONCEPTION FONCTIONNELLE	p. 8
05	CONCEPTION VISUELLE	p. 10
06	CONCEPTION TECHNIQUE	p. 12
07	ARCHITECTURE DE L'APPLICATION	p. 17
08	DÉVELOPPEMENT	p. 20
09	DÉPLOIEMENT ET CONTENEURISATION	p. 33
10	PLAN DE TESTS ET JEU D'ESSAI	p. 36
11	CONCLUSION	p. 38
	GLOSSAIRE	p. 40
	ANNEXES	p. 41

01

INTRODUCTION

Ce dossier présente le projet CLAP!, réalisé dans le cadre de la formation Développeur Web et Web Mobile (DWWM), titre professionnel de niveau 5 reconnu par l'État. La formation s'est déroulée en 2025-2026 et ce projet constitue le fil rouge servant de support à l'évaluation des compétences du référentiel.

CLAP! est une application web full-stack de découverte de films et de séries. Son nom est un clin d'œil au clap de cinéma, symbole du septième art. Le projet répond à une problématique concrète : avec la multiplication des plateformes de streaming, les utilisateurs passent en moyenne 18 minutes à chercher quoi regarder avant de faire un choix. CLAP! centralise la recherche, la notation, les commentaires et la gestion des favoris en une seule interface.

J'ai conçu et développé l'intégralité de l'application en autonomie, de l'analyse du besoin au déploiement. J'ai adopté une architecture microservices avec Spring Boot pour le backend, React 18 pour le frontend, et Docker pour le déploiement. Ce choix m'a permis d'illustrer les patterns architecturaux modernes et de démontrer la maîtrise de l'ensemble du cycle de développement.

02

COMPÉTENCES MISES EN ŒUVRE

CCP1 — Développer la partie front-end d'une application web sécurisée

Maquetter une application web mobile

J'ai conçu les wireframes et les maquettes de l'application sur Figma, en version desktop et mobile. J'ai appliqué les règles d'ergonomie (zones de chaleur, hiérarchie visuelle), les principes d'accessibilité (contrastes WCAG AA) et les règles RGPD (formulaire de consentement, politique de confidentialité). Un prototype interactif a été créé pour valider les parcours utilisateurs avant le développement.

Réaliser une interface utilisateur web statique et adaptable

J'ai développé l'ensemble du frontend en React 18 avec Tailwind CSS en approche mobile-first. La palette de couleurs personnalisée (fond sombre #0D0D14, accent or #C9A96E) assure la cohérence visuelle. Les breakpoints Tailwind (sm, md, lg, xl) garantissent l'adaptation sur tous les formats d'écran.

Développer une interface utilisateur web dynamique

J'ai développé une Single Page Application (SPA) avec React Router v6. Les interactions utilisateurs (favoris, notes, commentaires, notifications) sont gérées dynamiquement via TanStack Query pour l'état serveur et Zustand pour l'état global. Les animations (Framer Motion, GSAP, Three.js) enrichissent l'expérience sans nuire aux performances. L'internationalisation (i18next) supporte le français et l'anglais.

CCP2 — Développer la partie back-end d'une application web sécurisée

Créer une base de données

J'ai conçu la base de données relationnelle en suivant la méthode Merise (MCD → MLD → MPD). La base H2 en mode MySQL comporte 10 tables avec clés primaires, clés étrangères et contraintes d'intégrité. Les migrations sont versionnées avec Flyway. J'ai également utilisé MongoDB pour le cache des réponses TMDb.

Développer les composants d'accès aux données

J'ai développé les composants d'accès aux données en Java avec Spring Data JPA et Hibernate pour la base relationnelle, et MongoTemplate pour MongoDB. Un db-service centralisé expose une API REST interne consommée par les autres microservices, évitant le couplage direct à la base de données.

Développer la partie back-end d'une application web sécurisée

J'ai développé cinq microservices Spring Boot autonomes : auth-service, movies-service, social-service, notifications-service et db-service. Chaque service applique les principes REST, valide les entrées côté serveur (@Valid, @NotNull, @Size) et retourne des réponses HTTP normalisées. La sécurité repose sur JWT (HMAC-SHA256) et BCrypt pour le hachage des mots de passe.

Contribuer à la gestion d'un projet informatique

J'ai géré l'intégralité du cycle de développement en autonomie. J'ai planifié les grandes étapes (conception, développement, tests, déploiement) et les ai déclinées en user stories avec critères d'acceptation. J'ai utilisé Git avec des branches fonctionnelles et des pull requests pour maintenir un historique propre et une qualité de code constante.

03

RÉSUMÉ DU PROJET

Concept

CLAP! est une plateforme web de découverte de films et séries, destinée à centraliser la gestion des contenus audiovisuels. Elle s'adresse aux cinéphiles et amateurs de séries souhaitant disposer d'un espace personnel pour rechercher, noter, commenter et sauvegarder des contenus.

Le projet part d'un constat simple : l'offre de streaming s'est multipliée (plateformes généralistes, catalogues spécialisés, contenus exclusifs), mais aucune d'entre elles ne couvre l'ensemble du catalogue ni n'offre d'espace social dédié à la critique et au suivi personnel. Un utilisateur doit ainsi jongler entre plusieurs applications pour rechercher un contenu, vérifier où il est disponible, lire des avis fiables et garder une trace de ce qu'il a déjà vu ou souhaite voir. CLAP! répond à ce besoin de centralisation en s'appuyant sur l'API TMDB pour la donnée brute (catalogue, casting, plateformes de diffusion) et en y ajoutant une couche sociale propre : notes, commentaires, listes personnalisées et historique.

Sur le plan technique, le projet a été pensé dès la conception comme une démonstration de compétences DWWM plutôt que comme un simple prototype : architecture en cinq microservices Spring Boot autour d'une Gateway et d'un registre Consul, frontend React 18 en Single Page Application, authentification JWT avec gestion des rôles, persistance hybride (base relationnelle H2 pour les données métier, MongoDB pour le cache TMDB) et déploiement entièrement conteneurisé via Docker Compose. Chaque choix technique — détaillé dans les sections suivantes — répond à une compétence précise du référentiel : maquettage et accessibilité, développement front-end statique et dynamique, conception de base de données, développement back-end sécurisé et gestion de projet.

Fonctionnalités

Un visiteur non connecté peut :

- Découvrir l'application via la page d'accueil (animations 3D, présentation du concept)
- Rechercher des films et séries via le catalogue
- Consulter les fiches détaillées (synopsis, casting, bande-annonce, plateformes de streaming)
- Créer un compte et s'authentifier
- Consulter les mentions légales et la politique de confidentialité

Un utilisateur connecté peut en plus :

- Gérer son profil (avatar, bio, langue, thème, bannière personnalisée)
- Ajouter des contenus à ses favoris
- Créer des listes personnalisées publiques ou privées
- Noter des films et séries (1 à 10)

- Poster des commentaires et liker les commentaires d'autres utilisateurs
- Consulter son historique de navigation
- Recevoir des notifications (badges, alertes de sortie, réponses)
- Découvrir des contenus via le mode Discovery (filtres par humeur / genre)

Un administrateur connecté peut en plus :

- Consulter le tableau de bord avec les statistiques de la plateforme
- Gérer les utilisateurs (liste, promotion, suppression)
- Modérer les commentaires et les notes
- Consulter les journaux d'audit des actions administratives
- Envoyer des notifications à l'ensemble des utilisateurs

Cette progression par niveau d'accès — visiteur, utilisateur, administrateur — structure l'ensemble du projet : elle a guidé la rédaction des user stories, la conception du modèle de données (table users centrale, rôle stocké et vérifié à chaque requête) et la mise en place de la sécurité applicative présentée dans les sections suivantes.

04

CONCEPTION FONCTIONNELLE

Cahier des charges et priorisation MoSCoW

Pour commencer la conception, j'ai rédigé un cahier des charges formalisant l'objectif de l'application, les fonctionnalités souhaitées et leur classification par niveau de priorité selon la méthode MoSCoW :

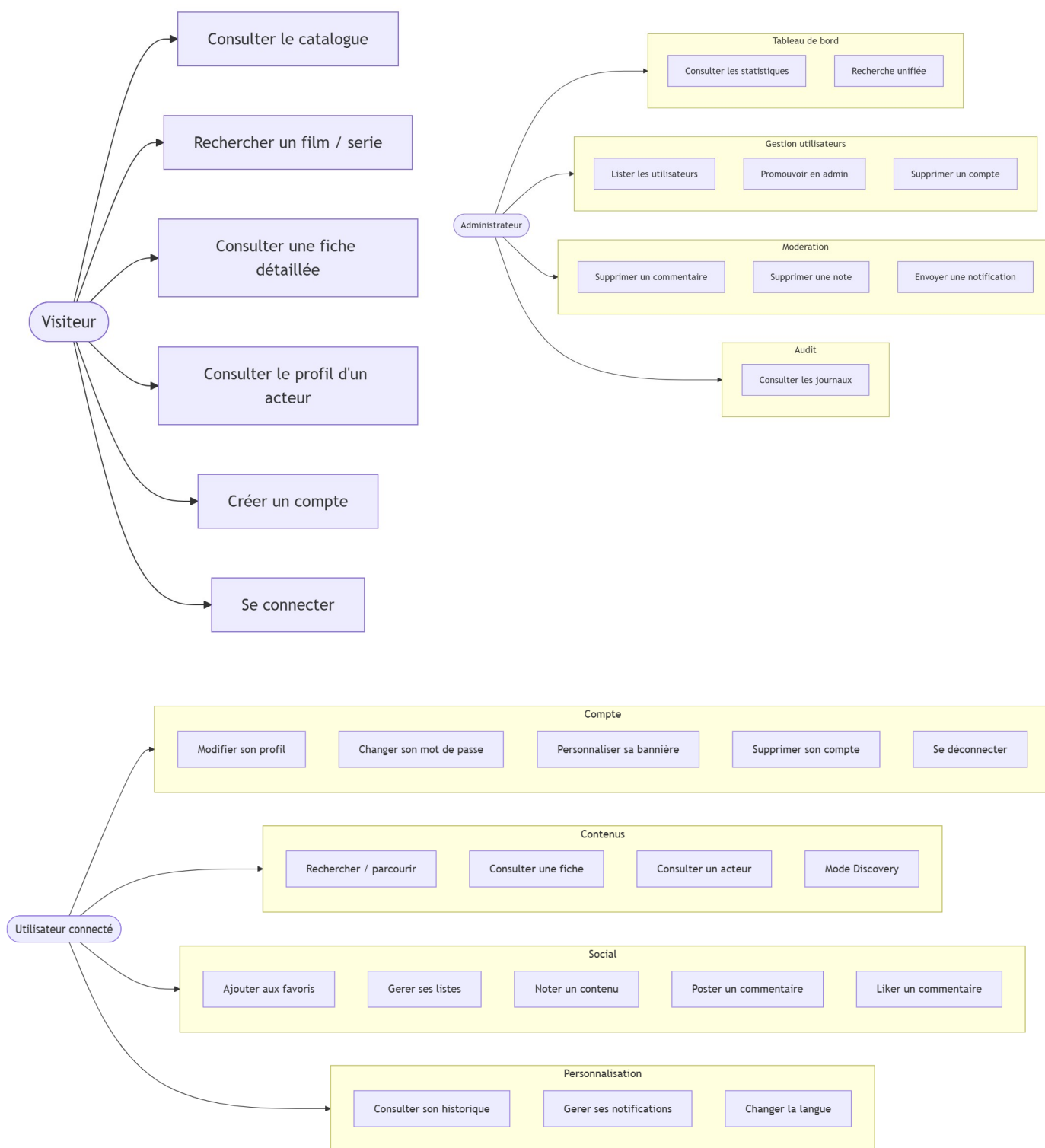
Priorité	Fonctionnalités
Must have	Authentification (inscription/connexion), recherche TMDB, fiches détaillées, favoris, notation
Should have	Commentaires, listes personnalisées, notifications, profil enrichi, historique
Could have	Découverte par humeur, internationalisation, gamification (XP/badges), bannière personnalisée
Won't have	Recommandations IA, applications mobiles natives, paiements en ligne

User stories et backlog

À partir du cahier des charges, j'ai formalisé un backlog sous forme de document récapitulant les users stories avec leurs critères d'acceptation et leurs priorité MoSCoW

US-01	Création de compte	Visiteur	● Must have	5
US-02	Connexion	Utilisateur inscrit	● Must have	3
US-03	Déconnexion	Utilisateur connecté	● Must have	1
US-04	Modification du profil	Utilisateur connecté	● Should have	3
US-05	Bannière personnalisée	Utilisateur connecté	● Could have	3
US-06	Changement de mot de passe	Utilisateur connecté	● Should have	2

Diagramme des cas d'utilisation



05

CONCEPTION VISUELLE

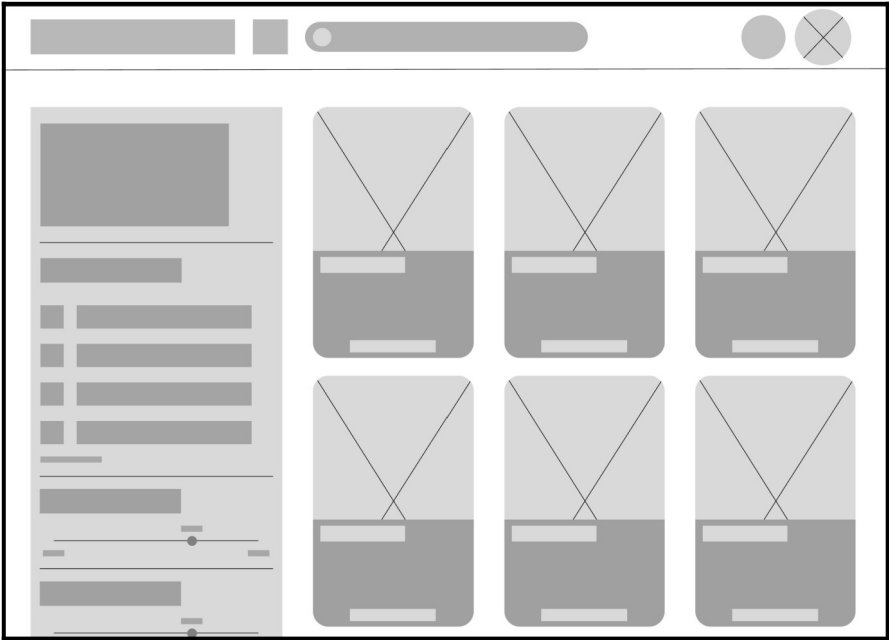
Charte graphique

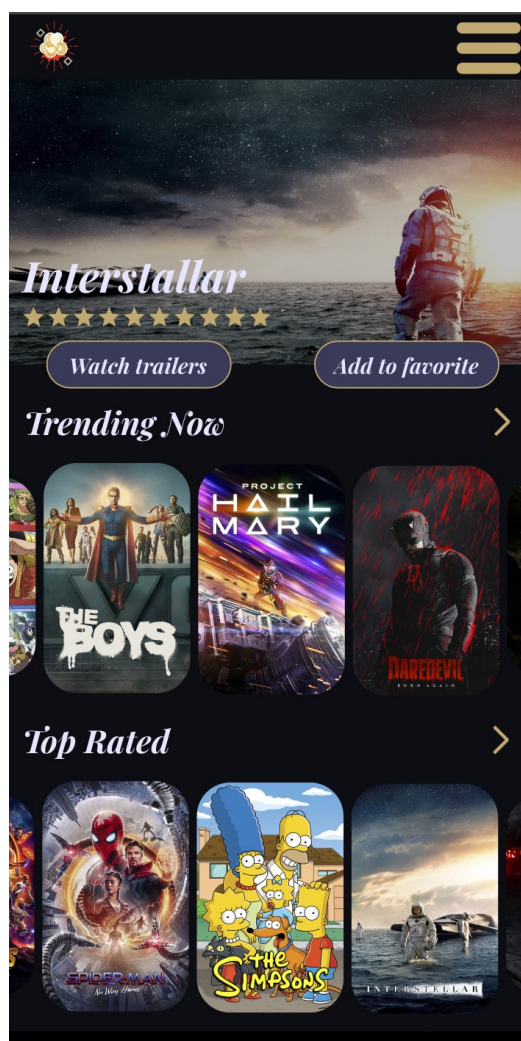
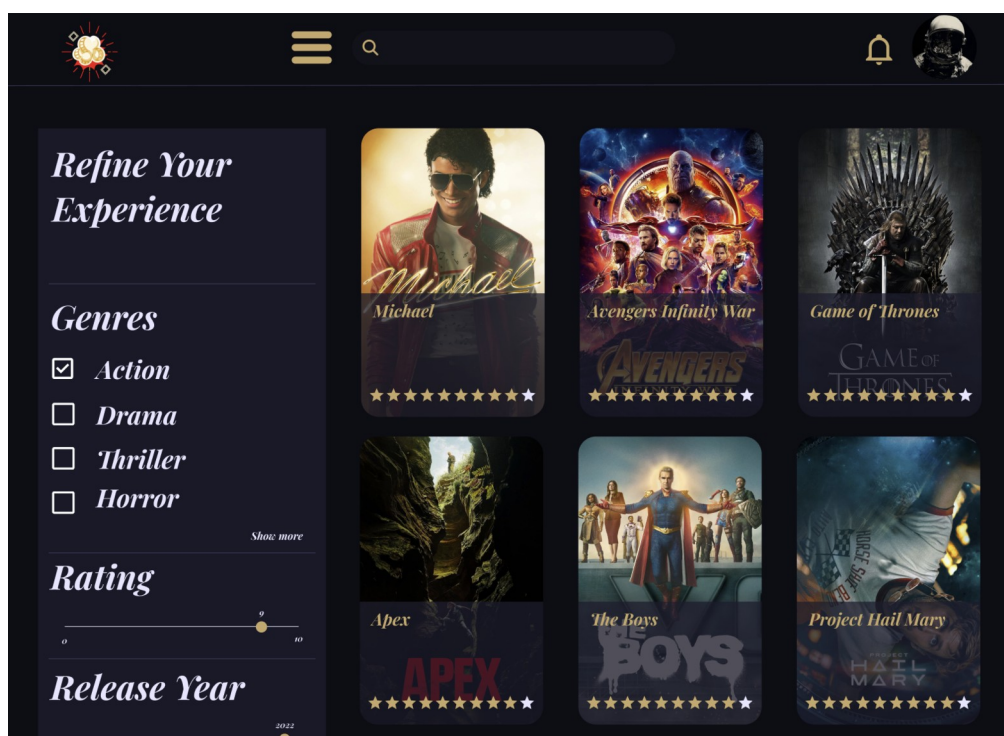
Palette de couleurs

Nom	Valeur hex	Utilisation
clap-bg	#0D0D14	Fond principal de l'application
clap-dark	#12121A	Fond des cartes et sections
clap-card	#1E1E2E	Composants de surface (cards, modals)
clap-gold	#C9A96E	Couleur d'accent — titres, CTA, bordures
clap-light	#E8E6FF	Textes secondaires sur fond sombre
clap-green	#4CAF7D	États positifs (succès, actif)
clap-red	#E05252	États d'erreur et d'alerte

Wireframes et maquettes

J'ai élaboré des wireframes puis des maquettes haute-fidélité sur Figma pour les versions desktop et mobile de l'application. Un prototype interactif a été créé pour représenter la navigation et valider les parcours utilisateurs avant le développement.





06

CONCEPTION TECHNIQUE

Environnement technique

Backend — Java & Spring Boot

Le backend de CLAP! repose sur Java 17 et Spring Boot 3.5. Ce choix s'impose pour ses nombreux avantages : serveur Tomcat embarqué, configuration automatisée, injection de dépendances via annotations (`@RestController`, `@Service`, `@Repository`), et accès aux données simplifié avec Spring Data JPA / Hibernate.

Spring Cloud enrichit l'architecture microservices avec Spring Cloud Gateway (routage réactif, filtres JWT, CORS centralisé), Spring Cloud Consul (découverte dynamique des services) et Resilience4j (circuit breaker, retry). La gestion des dépendances est assurée par Maven via les fichiers pom.xml.

Frontend — React 18 & Vite

J'ai choisi React 18 pour sa maturité, sa communauté et sa compatibilité avec les exigences DWWM. Vite remplace Create React App pour sa rapidité de build et son Hot Module Replacement ultra-rapide. Tailwind CSS est utilisé en mobile-first avec une palette personnalisée. La gestion d'état repose sur Zustand (état global d'authentification) et TanStack Query (état serveur).

Bases de données

Les données sont réparties entre deux systèmes : H2 (mode MySQL) pour les données relationnelles métier, géré exclusivement par le db-service, et MongoDB pour le cache des réponses TMDB. H2 en mode fichier persistant garantit la compatibilité syntaxique MySQL pour une migration future sans reconfiguration.

Technologies utilisées

Catégorie	Technologie	Rôle
Langage backend	Java 17	Langage de programmation principal
Backend	Spring Boot 3.5	Framework applicatif microservices
Backend	Spring Cloud Gateway	API Gateway (routage, JWT, CORS)
Backend	Spring Security + JJWT 0.12.5	Authentification JWT / RBAC
Backend	Spring Data JPA / Hibernate	ORM et accès base de données
Backend	OpenFeign	Clients HTTP déclaratifs inter-services
Backend	Resilience4j	Circuit breaker et retry

Catégorie	Technologie	Rôle
Backend	Flyway	Migrations de schéma versionnées
Backend	MapStruct 1.5.5	Mapping DTO automatisé
Langage frontend	JavaScript (ES2023)	Langage frontend
Frontend	React 18.2	Bibliothèque UI
Frontend	Vite 5.0	Bundler et serveur de développement
Frontend	Tailwind CSS 3.4	Framework CSS utilitaire
Frontend	React Router v6	Routing SPA
Frontend	Zustand 4.5	Gestion d'état global
Frontend	TanStack Query 5.17	État serveur et cache côté client
Frontend	Axios 1.6	Client HTTP avec intercepteurs JWT
Frontend	i18next 23.8	Internationalisation (fr / en)
Frontend	Framer Motion 11	Animations React
Frontend	Three.js / React Three Fiber	Effets 3D (page d'accueil)
Frontend	GSAP 3.14	Animations timeline avancées
BDD relationnelle	H2 (mode MySQL)	Base de données applicative persistante
BDD NoSQL	MongoDB 7	Cache des réponses API TMDB
Conteneurisation	Docker / Docker Compose	Conteneurs et orchestration
Discovery	Consul (HashiCorp)	Registre de services dynamique
Serveur web	Nginx (Alpine)	Serveur frontend + proxy API
IDE	IntelliJ IDEA / VS Code	Environnements de développement
Versioning	Git / GitHub	Gestion de version et pull requests
Tests	JUnit 5 / Vitest	Tests unitaires backend et frontend
Documentation API	Swagger UI	Documentation des endpoints REST

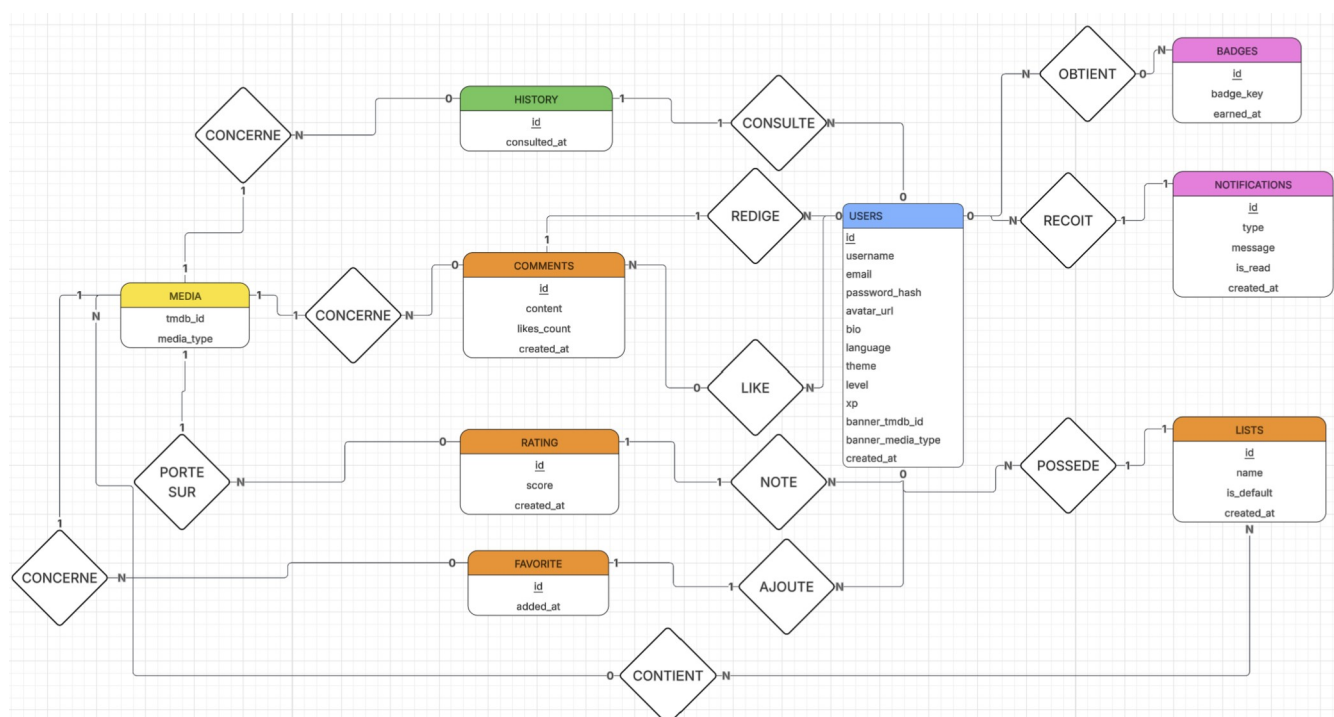
Modélisation et conception de la base de données

Avant de créer la base de données, j'ai suivi la méthode Merise : MCD → MLD → MPD. Cette démarche m'a permis de structurer la base avec cohérence et de définir précisément les relations entre les entités. La base principale H2 contient 10 tables liées à l'entité centrale utilisateur. En parallèle, MongoDB gère le cache des données TMDB.

Modèle Conceptuel de Données (MCD)

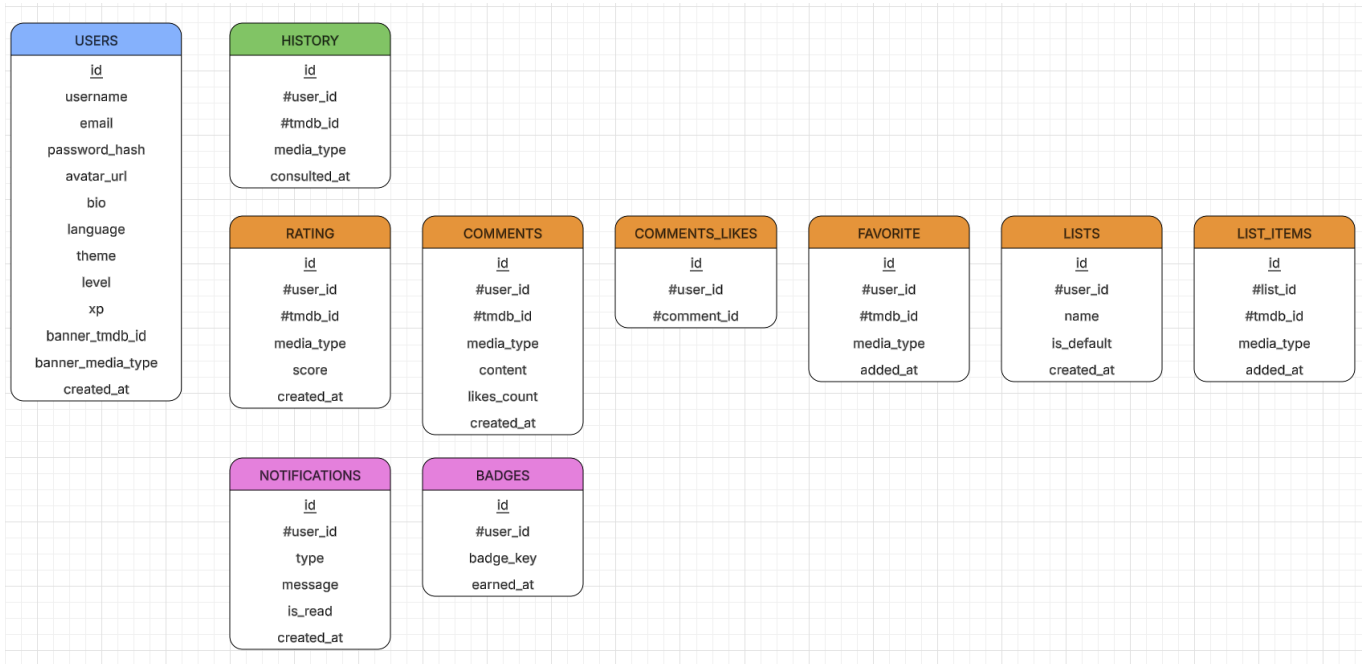
Le MCD représente les entités fondamentales de CLAP! et leurs associations :

Entité	Description
users	Comptes et profils utilisateurs (rôle, XP, niveau, préférences)
favorites	Contenus TMDB mis en favoris par un utilisateur
comments	Commentaires postés sur les fiches films/séries
comment_likes	Réactions (likes) sur les commentaires (relation many-to-many)
ratings	Notes (1-10) attribuées par un utilisateur à un contenu TMDB
lists	Listes/playlists personnalisées (publiques ou privées)
list_items	Contenus ajoutés dans une liste
history	Historique des fiches consultées par un utilisateur
notifications	Notifications de la plateforme (badge, alerte, réponse...)
admin_logs	Journal d'audit des actions des administrateurs



Modèle Logique de Données (MLD)

Pour le MLD, chaque entité devient une table et son identifiant est transformé en clé primaire. Les associations many-to-many (users ↔ listes, commentaires ↔ likes) génèrent des tables de liaison. Les associations one-to-many sont représentées par des clés étrangères. Par exemple, la table comments comporte une clé primaire id ainsi que les clés étrangères userId et tmdbId.



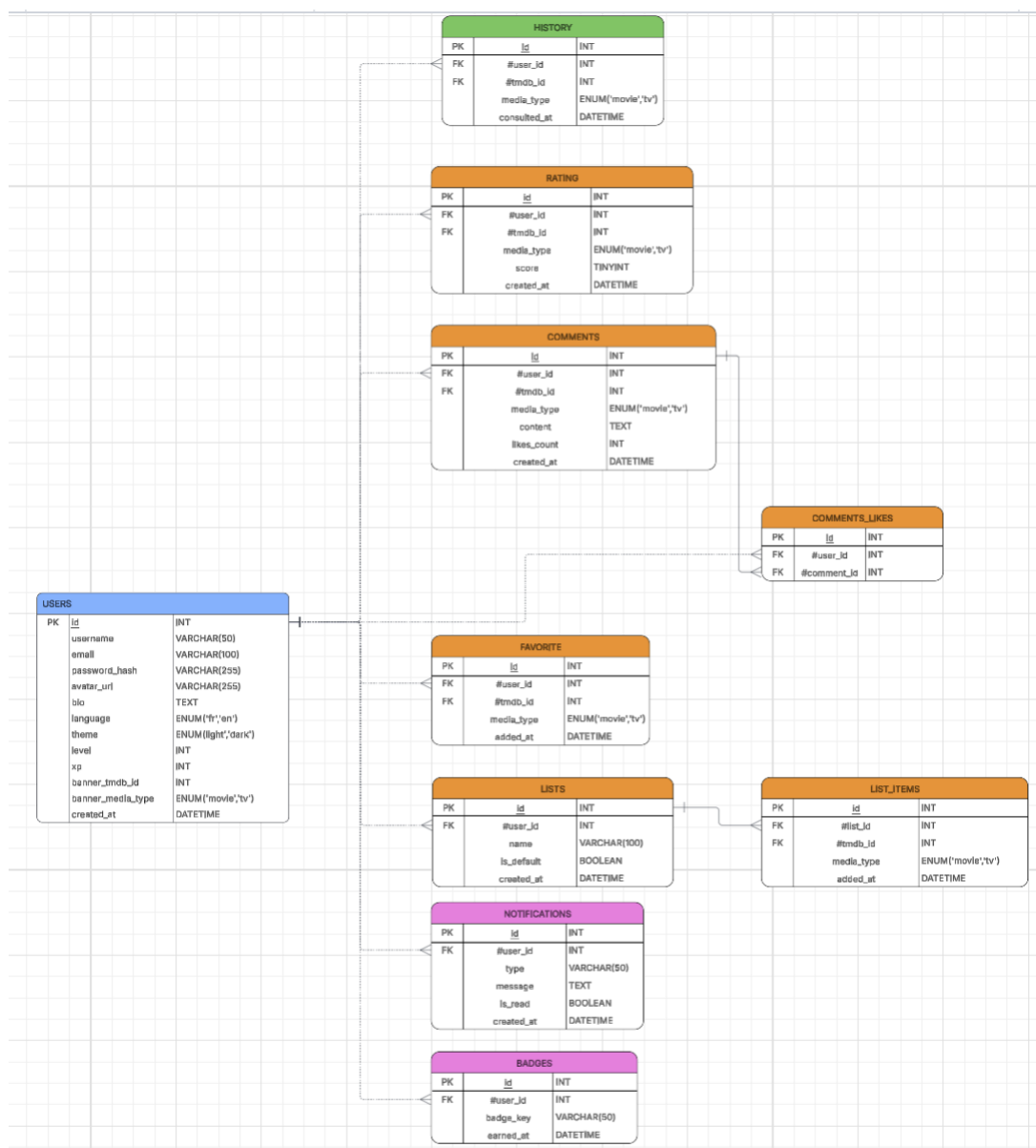
Modèle Physique de Données (MPD)

Pour le MPD, j'ai utilisé H2 en mode MySQL. J'ai choisi les types de données adaptés (INT, VARCHAR, TEXT, TINYINT, BOOLEAN, DATETIME). J'ai implémenté les clés primaires (PRIMARY KEY NOT NULL AUTO_INCREMENT), les clés étrangères avec les actions appropriées, et les contraintes d'intégrité de domaine (UNIQUE sur email et username, CHECK sur score entre 1 et 10).

Exemple : dans la table comment_likes, la clé étrangère commentId utilise ON DELETE CASCADE pour supprimer automatiquement les likes lorsqu'un commentaire est supprimé. La contrainte UNIQUE sur (userId, commentId) empêche les doublons de likes.

```

CREATE TABLE comment_likes (
  id          BIGINT          NOT NULL AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT          NOT NULL,
  comment_id  BIGINT          NOT NULL,
  created_at  DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY  uq_user_comment (user_id, comment_id),
  FOREIGN KEY (comment_id) REFERENCES comments(id) ON DELETE CASCADE,
  FOREIGN KEY (user_id)   REFERENCES users(id)   ON DELETE CASCADE
);
  
```

Configuration de la base de données et sécurité

La base H2 est déployée dans le conteneur db-service avec un volume Docker persistant (/data). Le fichier de configuration application.yaml définit : l'URL JDBC (jdbc:h2:file:/data/clap;MODE=MySQL), le dialecte Hibernate compatible MySQL, et Flyway pour les migrations automatiques au démarrage. Trois migrations versionnées assurent l'évolution contrôlée du schéma :

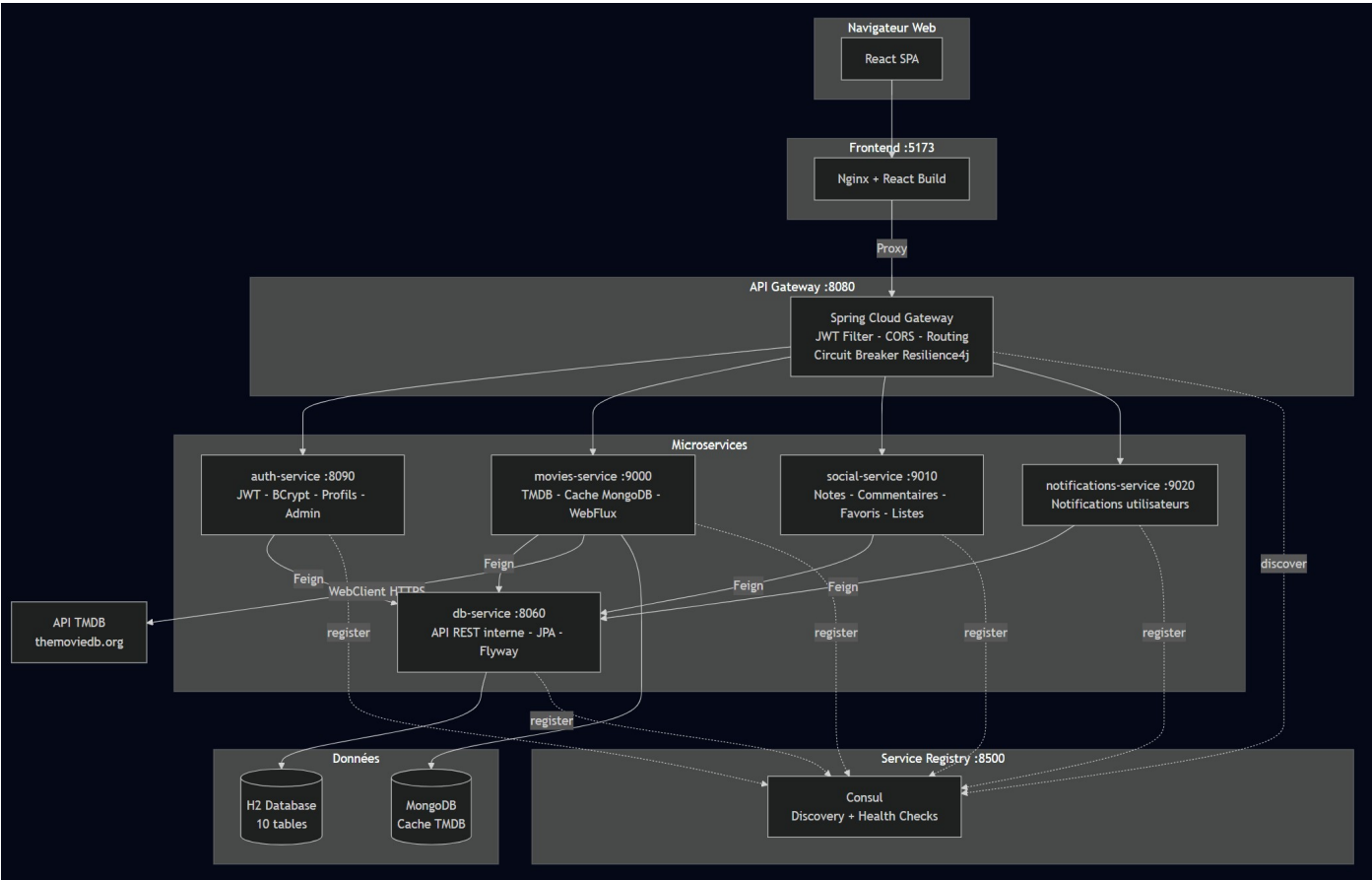
- V1__init_all.sql : création de toutes les tables initiales
- V2__add_user_role.sql : ajout de la colonne role (user/admin)
- V3__add_admin_logs.sql : création de la table admin_logs

07

ARCHITECTURE DE L'APPLICATION

Description générale de l'architecture

L'architecture de CLAP! est une architecture distribuée en microservices. Les différentes responsabilités (présentation, authentification, médias, social, notifications, persistance) sont réparties entre des services autonomes et indépendants, plutôt que centralisées dans un monolithe. Chaque microservice peut évoluer, être déployé et redimensionné indépendamment.



Les microservices

Service	Port	Rôle principal
auth-service	8090	Authentification JWT, profils, gestion des utilisateurs et administration
movies-service	9000	Proxy TMDb, cache MongoDB, recherche, fiches détaillées, acteurs

Service	Port	Rôle principal
social-service	9010	Notes, commentaires, likes, favoris, listes personnalisées
notifications-service	9020	Création et gestion des notifications utilisateurs
db-service	8060	Service centralisé d'accès à la base H2 (API REST interne)
gateway	8080	Routage, validation JWT, CORS, circuit breaker Resilience4j
frontend (Nginx)	5173	SPA React + proxy API vers la Gateway

Design patterns utilisés

API Gateway Pattern

La Gateway est le seul point d'entrée externe de l'API. Elle centralise la validation JWT, le routage vers les services, la configuration CORS et les circuit breakers. Cela évite de dupliquer ces logiques dans chaque microservice et simplifie les évolutions de sécurité.

Service Registry & Discovery

Consul joue le rôle de registre de services. À chaque démarrage, chaque service Spring Boot s'enregistre automatiquement dans Consul avec son nom, son port et son endpoint de health check (/actuator/health). La Gateway résout les URLs via lb://[nom-du-service], permettant le load balancing dynamique sans configuration statique.

Circuit Breaker Pattern

Resilience4j protège contre les cascades de pannes. Le movies-service utilise un circuit breaker pour les appels à l'API TMDb : si 50% des appels échouent sur une fenêtre de 10 requêtes, le circuit s'ouvre et les requêtes sont immédiatement rejetées avec HTTP 503, sans attendre le timeout de 3 secondes.

Layered Architecture (interne à chaque service)

Chaque microservice est organisé en couches : Controller (réception des requêtes HTTP), Service (logique métier), Repository (accès aux données via JPA/Hibernate), Model (entités et DTOs). Cette séparation des responsabilités facilite la maintenabilité et la testabilité.

Database per Service

Seul le db-service interagit directement avec la base H2. Les autres services communiquent avec les données exclusivement via l'API REST du db-service (clients OpenFeign). Ce pattern évite le couplage direct aux données et permet d'évoluer ou de migrer la base sans impacter les services consommateurs.

Communication entre microservices

La communication est synchrone et basée sur HTTP/REST. Chaque service appelle les autres via des interfaces OpenFeign annotées `@FeignClient`. La Gateway injecte les headers utilisateur (X-User-Id, X-User-Email, X-User-Role) après validation du JWT, et les services aval les consomment sans re-valider le token.

Exemple de flux — Ajout d'un commentaire :

- L'utilisateur soumet le formulaire depuis React
- Axios envoie POST /comments avec le JWT dans Authorization: Bearer
- La Gateway valide le JWT, injecte X-User-Id et route vers le social-service
- Le social-service crée le commentaire via le db-service (OpenFeign)
- Le notifications-service est appelé pour créer une notification si nécessaire
- La réponse HTTP 201 est propagée jusqu'au frontend

Les services internes (db-service, notifications-service) ne sont jamais exposés à l'extérieur. Seule la Gateway est accessible depuis le réseau externe.

08

DÉVELOPPEMENT

Authentification et autorisation

La sécurité de CLAP! repose sur JWT (JSON Web Token, RFC 7519) et Spring Security. Le token est signé avec HMAC-SHA256 et une clé de 256 bits fournie via variable d'environnement (JWT_SECRET). Sa durée de validité est de 24 heures.

Flux d'authentification

- L'utilisateur envoie ses identifiants (email/password) via POST /auth/login
- L'auth-service récupère l'utilisateur depuis le db-service via OpenFeign
- BCrypt compare le mot de passe saisi au hash stocké en base
- Si valide, un JWT est généré contenant { userId, email, role, iat, exp }
- Le frontend stocke le token dans localStorage via Zustand (persist middleware)
- Axios injecte automatiquement Authorization: Bearer <token> sur chaque requête

```
// Payload JWT
{
  "sub": "user@exemple.fr",
  "userId": 42,
  "role": "user",
  "iat": 1700000000,
  "exp": 1700086400 // iat + 24h
}
```

Gestion des rôles (RBAC)

Deux rôles existent : user (défaut) et admin. Le rôle est encodé dans le JWT. La Gateway vérifie le rôle avant les routes /admin/** et retourne HTTP 403 si insuffisant. Le composant AdminRoute côté React empêche l'affichage des pages admin aux non-administrateurs.

```
// JwtAuthFilter.java (Gateway) - vérification du rôle admin
if (path.startsWith("/admin") && !"admin".equals(role)) {
  response.setStatus(HttpStatus.FORBIDDEN);
  return response.setComplete();
}
```

Validation des données

Avant toute persistance, les données sont validées côté serveur via les annotations Jakarta Validation (`@NotNull`, `@NotBlank`, `@Size`, `@Email`, `@Min`, `@Max`). L'annotation `@Valid` sur les paramètres de contrôleur déclenche la validation automatiquement. En cas d'erreur, Spring retourne une réponse HTTP 400 avec les messages d'erreur détaillés.

```
// RegisterRequest.java - exemple de validation
public class RegisterRequest {
    @NotBlank @Size(min=3, max=30)
    private String username;

    @NotBlank @Email
    private String email;

    @NotBlank @Size(min=8)
    private String password;
}
```

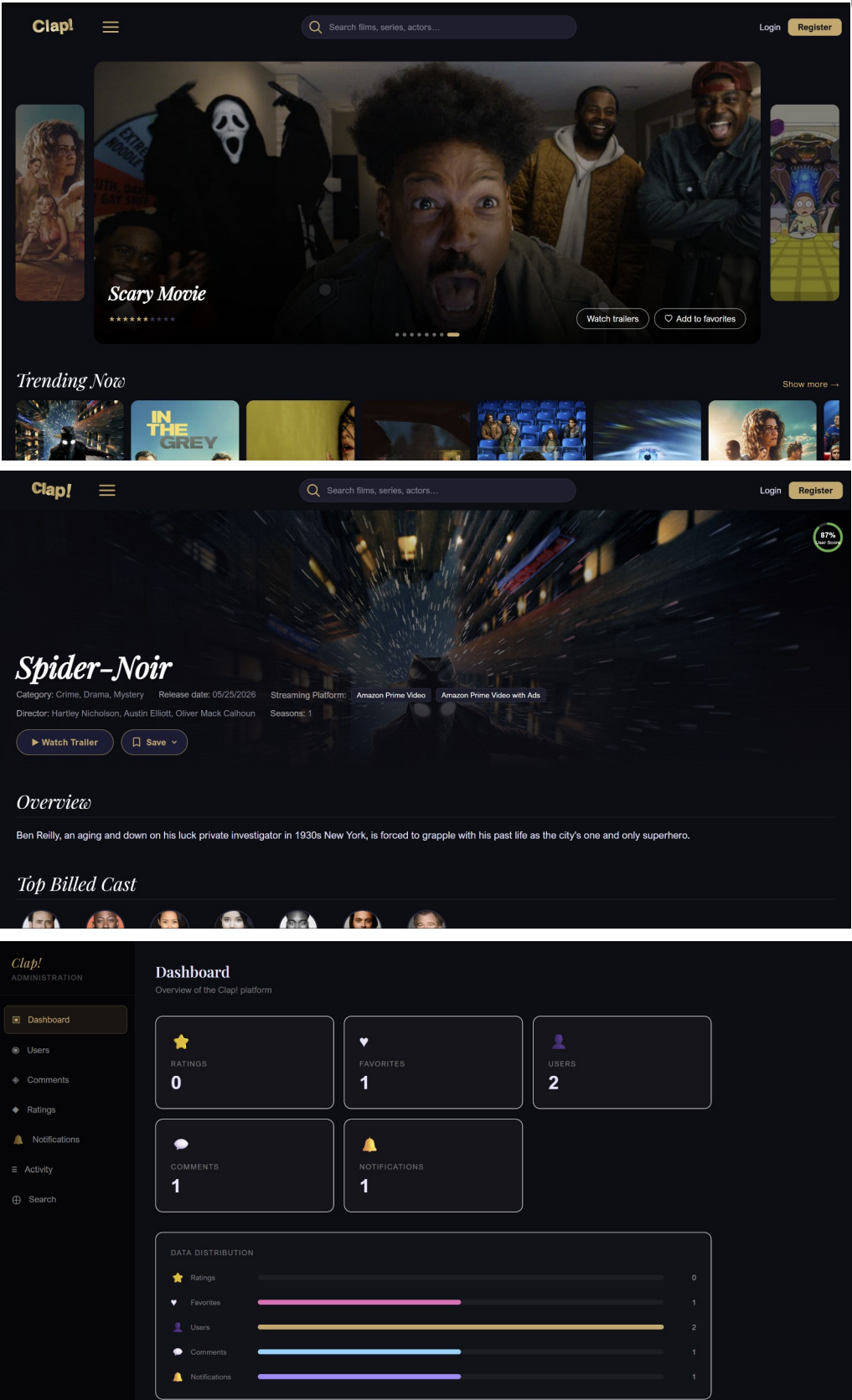
Cache TMDB avec MongoDB

Le movies-service utilise MongoDB pour mettre en cache les réponses JSON de l'API TMDB. Chaque document est indexé avec un TTL automatique (configurable par type de requête). La clé de cache est déterministe et inclut tous les paramètres (tmdbId, mediaType, language). Un circuit breaker Resilience4j protège les appels TMDB : seuil d'échec 50%, fenêtre 10 appels, timeout 3 secondes.

```
// TmdbService.java - appel avec cache et circuit breaker
@CircuitBreaker(name = "tmdb", fallbackMethod = "fallback")
@Cacheable(value = "tmdb_details", key = "#tmdbId + '_' + #mediaType + '_' + #language")
public Mono<TmdbDetails> getDetails(Long tmdbId, String mediaType, String language) {
    return webClient.get()
        .uri("/movie/{id}?language={lang}", tmdbId, language)
        .retrieve().bodyToMono(TmdbDetails.class);
}
```

Frontend — Architecture React

Le frontend est organisé en couches : pages (composants de haut niveau), composants/layout (Navbar, Footer, ProtectedRoute), composants/ui (cartes, toasts, modals), store (Zustand), services (Axios), hooks (logique réutilisable) et locales (traductions i18n).



Structure du projet React

Le projet est organisé en dossiers sémantiques pour une lisibilité et une maintenabilité maximales :

Dossier	Contenu	Rôle
src/pages/	26 pages React (Home, Catalogue, MovieDetail...)	Composants de niveau page
src/components/layout/	Navbar, Footer, ProtectedRoute, AdminRoute...	Composants structurels
src/components/ui/	MovieCard, Skeleton, Button, Spinner...	Composants réutilisables
src/components/film/	Carousel, MovieCard	Composants spécifiques TMDB
src/store/	authStore.js, notificationStore.js	Stores Zustand (état global persistant)
src/services/	api.js, authService.js, moviesService.js...	Configuration Axios et appels API
src/hooks/	useMovies.js, useSocial.js, useNotifications.js	Hooks React personnalisés
src/locales/	fr.json, en.json (~18 Ko chacun)	Fichiers de traduction i18next
src/utils/	formatters.js	Fonctions utilitaires (dates, formatage)
src/pages/admin/	7 pages d'administration	Back-office avec layout propre

Routing et protection des routes

React Router v6 gère la navigation côté client. J'ai mis en place deux niveaux de protection des routes avec des composants dédiés :

ProtectedRoute — Authentification requise

Ce composant vérifie la présence d'un token valide dans le store Zustand avant d'afficher une page protégée. Si l'utilisateur n'est pas connecté, il est redirigé vers /login avec Navigate de React Router.

```
// ProtectedRoute.jsx
import { Navigate } from 'react-router-dom';
import useAuthStore from '../../store/authStore';

export default function ProtectedRoute({ children }) {
  const isAuthenticated = useAuthStore((s) => s.isAuthenticated);
  return isAuthenticated ? children : <Navigate to="/login" replace />;
}
```

AdminRoute — Rôle administrateur requis

Ce composant ajoute une vérification supplémentaire du rôle. Si l'utilisateur est connecté mais n'est pas admin, il est redirigé vers l'accueil. Double protection avec le filtre JWT de la Gateway.

```
// AdminRoute.jsx
export default function AdminRoute({ children }) {
  const user = useAuthStore((s) => s.user);
  const isAuthenticated = useAuthStore((s) => s.isAuthenticated);

  if (!isAuthenticated) return <Navigate to='/login' replace />;
  if (user?.role !== 'admin') return <Navigate to='/' replace />;

  return children;
}
```

Vérification de l'expiration du JWT

Dans App.jsx, un effet React vérifie l'expiration du token à chaque changement de route. Si le token est expiré, l'utilisateur est déconnecté silencieusement et redirigé vers /login.

```
// App.jsx - vérification expiration JWT à chaque navigation
useEffect(() => {
  const token = localStorage.getItem('token');
  if (!token) return;
  try {
    const payload = JSON.parse(atob(token.split('.')[1]));
    if (payload.exp && payload.exp * 1000 < Date.now()) {
      logout();
    }
  } catch { logout(); }
}, [logout]);
```


Gestion d'état — Zustand

Zustand gère l'état global d'authentification. J'ai choisi Zustand plutôt que Redux pour sa simplicité (pas de boilerplate), sa performance (re-renders sélectifs par selector) et son middleware persist qui synchronise avec le localStorage.

```
// authStore.js — Store Zustand avec persistance localStorage
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

function extractRoleFromToken(token) {
  if (!token) return 'user';
  try {
    return JSON.parse(atob(token.split('.')[1]))?.role ?? 'user';
  } catch { return 'user'; }
}

const useAuthStore = create(
  persist(
    (set, get) => ({
      token: null, user: null, isAuthenticated: false,

      login: (token, user) => {
        localStorage.setItem('token', token);
        const role = extractRoleFromToken(token);
        set({ token, user: { ...user, role }, isAuthenticated: true });
      },

      logout: () => {
        localStorage.removeItem('token');
        set({ token: null, user: null, isAuthenticated: false });
      },
    }),
    { name: 'clap-auth' }
  )
);
```

Client HTTP — Axios et intercepteurs

J'ai configuré une instance Axios centralisée dans src/services/api.js partagée par tous les services. Elle intègre deux intercepteurs essentiels :

Intercepteur de requête — Injection du JWT et de la langue

Avant chaque requête, l'intercepteur injecte le token JWT dans Authorization: Bearer. Pour les routes TMDB (/movies, /tv, /actors), il ajoute le paramètre language selon la langue active (fr-FR ou en-US).

```
// api.js - Intercepteur de requête
const TMDB_LOCALE = { fr: 'fr-FR', en: 'en-US' };
const TMDB_PREFIXES = ['/movies', '/actors', '/tv'];

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;

  const isTmdbRoute = TMDB_PREFIXES.some(p => config.url?.startsWith(p));
  if (isTmdbRoute) {
    const lang = i18n.language ?? 'fr';
    config.params = { ...config.params,
      language: TMDB_LOCALE[lang] ?? 'fr-FR' };
  }
  return config;
});
```

Intercepteur de réponse — Gestion du 401

En cas de réponse HTTP 401 (token expiré ou invalide), l'intercepteur efface le token, affiche un toast 'Session expirée' (via i18n), puis redirige vers /login après 1,5 secondes.

```
// api.js - Intercepteur de réponse (401)
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      const hadToken = !!localStorage.getItem('token');
      localStorage.removeItem('token');
      if (hadToken) {
        useNotificationStore.getState().add({
          type: 'session',
          title: t('notifications.sessionExpired'),
          duration: 4000,
        });
        setTimeout(() => { window.location.href = '/login'; }, 1500);
      }
    }
    return Promise.reject(error);
  }
);
```

TanStack Query — Gestion de l'état serveur

TanStack Query gère l'état serveur : cache des réponses API, revalidation automatique, gestion des états de chargement et d'erreur. Contrairement à Zustand (état client), TanStack Query est dédié aux données provenant du serveur.

- Cache automatique : fiche film déjà visitée affichée instantanément (staleTime : 5 min)
- Déduplication : si deux composants demandent la même fiche, un seul appel réseau
- Revalidation : favoris rafraîchis automatiquement après ajout/suppression
- States gérés automatiquement : isLoading, isError, data sans boilerplate

Convention de clés : ['movies', tmdbId, language] pour une invalidation précise. `queryClient.invalidateQueries(['favorites'])` après un ajout aux favoris.

Composants clés

MovieCard — Carte d'un contenu TMDB

MovieCard est l'élément de base de l'interface. Il affiche l'affiche, le titre, l'année et la note TMDB. Framer Motion gère l'animation au survol (scale + élévation avec spring). Les images utilisent `loading='lazy'` pour optimiser les performances.

```
// MovieCard.jsx - Carte avec animation Framer Motion
export default function MovieCard({ movie, type = 'movie' }) {
  return (
    <motion.div
      whileHover={{ scale: 1.04, y: -4 }}
      transition={{ type: 'spring', stiffness: 300, damping: 20 }}
    >
      <Link to={`/${type === 'tv' ? 'serie' : 'film'}/${movie.id}`}>
        <div className='relative rounded-xl overflow-hidden aspect-[2/3]'>
          <img
            src={`/${TMDB_IMG}/${movie.posterPath}`}
            alt={movie.title}
            loading='lazy'
          />
          <div className='absolute top-2 right-2 bg-clap-bg/80
            text-clap-gold text-xs font-bold px-2 py-1 rounded-
full'>
            {movie.voteAverage?.toFixed(1)}
          </div>
        </div>
        <p className='mt-2 text-sm text-clap-light truncate'>{movie.title}</p>
      </Link>
    </motion.div>
  );
}
```

Skeleton — États de chargement

Pour éviter les sauts de mise en page (layout shifts) pendant le chargement, les composants Skeleton occupent l'espace avec l'animation `animate-pulse` de Tailwind CSS.

```
// Skeleton.jsx
export default function Skeleton({ className = '' }) {
  return (
    <div className={`bg-clap-muted/40 animate-pulse rounded-lg ${className}`} />
  );
}

export function MovieCardSkeleton() {
  return (
    <div className='flex-shrink-0 w-36 md:w-44'>
      <Skeleton className='w-full aspect-[2/3]' />
      <Skeleton className='mt-2 h-4 w-3/4' />
      <Skeleton className='mt-1 h-3 w-1/2' />
    </div>
  );
}
```

Internationalisation — i18next

L'application supporte le français et l'anglais via i18next. La langue est initialisée depuis le `localStorage` (fr par défaut) et persistée à chaque changement. La langue est aussi injectée dans les appels TMDB via l'intercepteur Axios.

```
// i18n.js
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import fr from './locales/fr.json';
import en from './locales/en.json';

i18n.use(initReactI18next).init({
  resources: { fr: { translation: fr }, en: { translation: en } },
  lng: localStorage.getItem('clap-lang') || 'fr',
  fallbackLng: 'fr',
  interpolation: { escapeValue: false },
});

i18n.on('languageChanged', (lng) => {
  localStorage.setItem('clap-lang', lng);
});
```

Animations et effets visuels

Framer Motion — Animations React

Framer Motion anime les interactions : scale au survol (whileHover), transitions de pages (AnimatePresence), apparitions progressives (variants + staggerChildren). Respect de prefers-reduced-motion.

Three.js / React Three Fiber — Effet 3D

La page d'accueil intègre une scène 3D avec React Three Fiber : objets géométriques (sphères, tores) flottants en orbite, désactivée automatiquement sur mobile pour préserver les performances.

GSAP — Animations timeline

GSAP gère les animations séquentielles complexes : apparition du titre CLAP!, déplacement des carrousels avec inertie, transitions des sections de la page d'accueil.

Responsive Design — Tailwind CSS

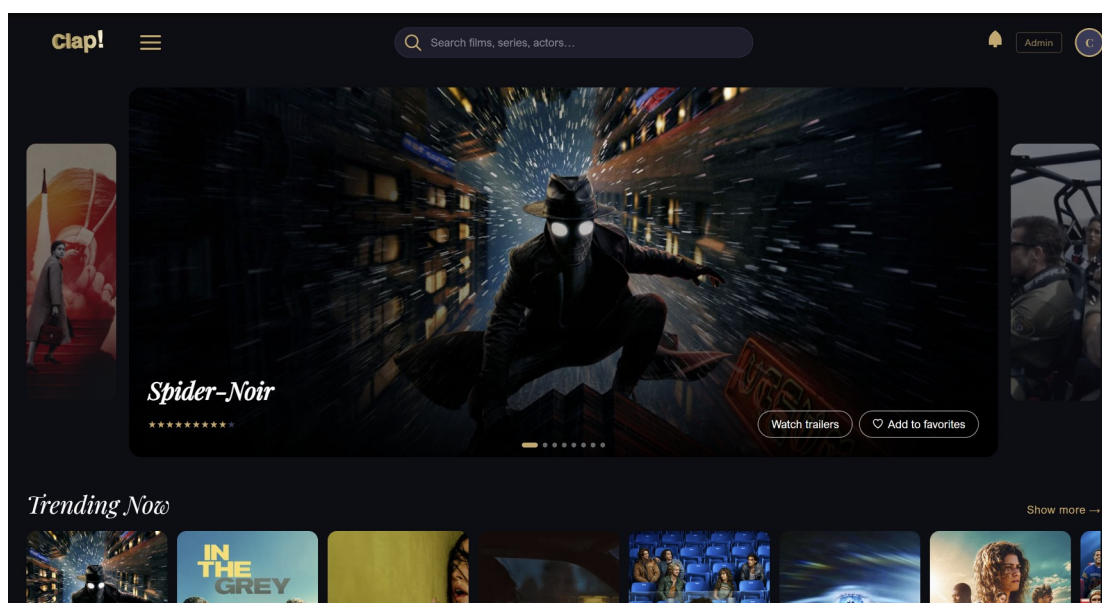
Interface développée en mobile-first avec Tailwind CSS. Adaptation aux quatre breakpoints sm/md/lg/xl.

Élément	Mobile	Tablette (md)	Desktop (lg/xl)
Grille de cartes	1-2 colonnes	3 colonnes	4-5 colonnes
Navigation	Menu hamburger	Menu hamburger	Barre horizontale complète
Fiche détaillée	Layout vertical	Layout vertical	2 colonnes (affiche + infos)
Carrousels	Scroll horizontal tactile	Scroll horizontal	Flèches de navigation
Taille des cartes	w-36 (144px)	w-44 (176px)	w-44 (176px)

Pages principales

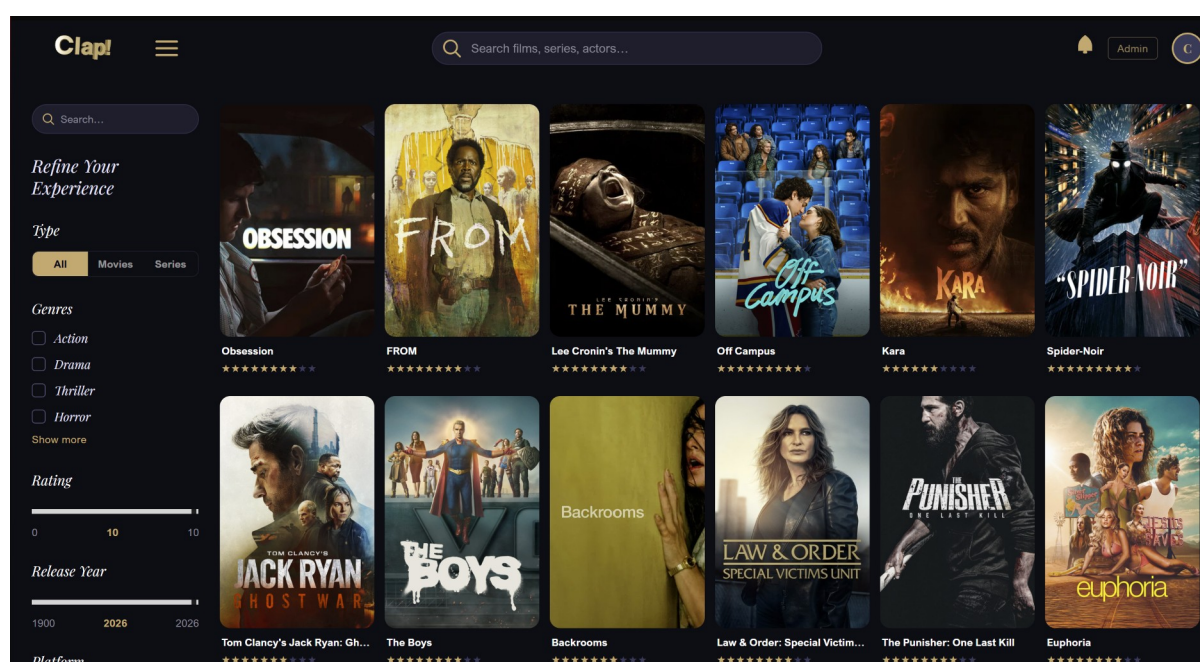
Page d'accueil (Home)

Section hero avec effet 3D Three.js pour le logo seulement et animation GSAP, carrousel de films tendances, carrousel de séries populaires, section de découverte par genre. Design sombre avec accents or.



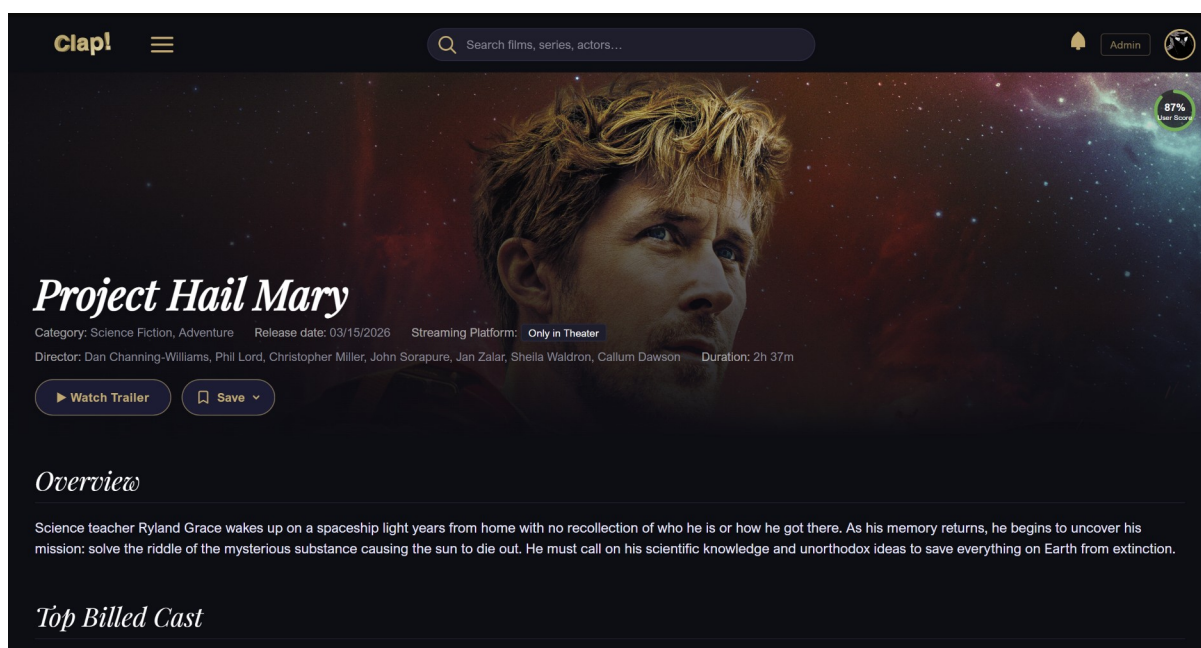
Catalogue

Onglets Films / Séries / Tendances, pagination, barre de recherche intégrée. MovieCardSkeleton pendant le chargement pour éviter les sauts visuels.



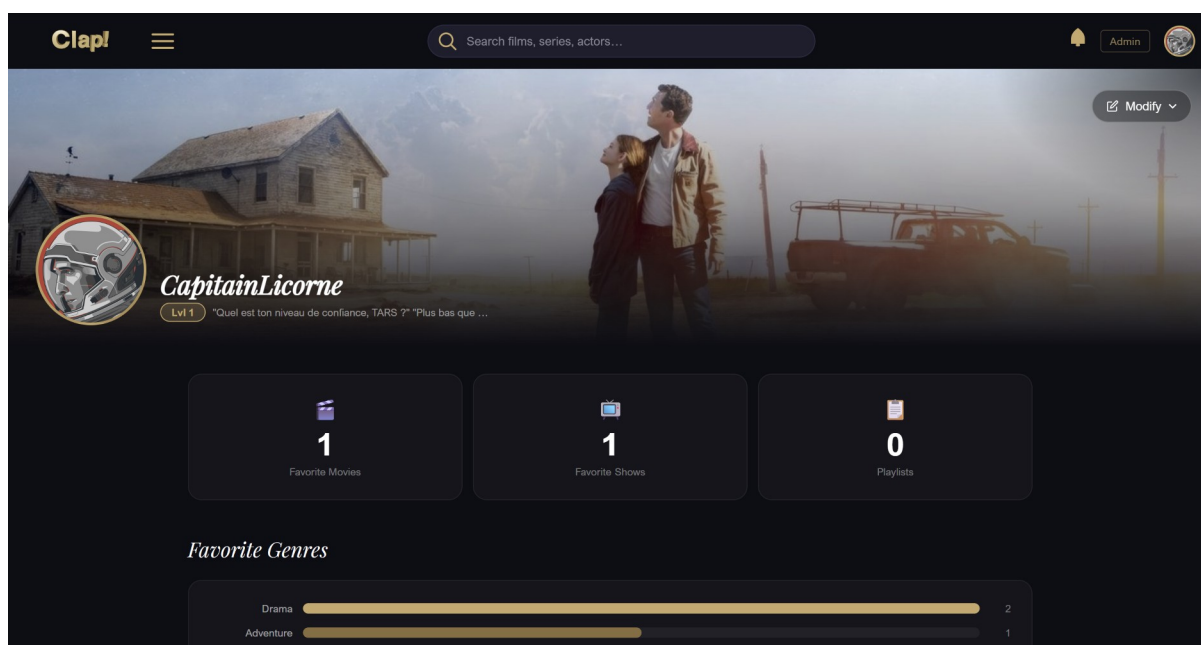
Fiche détaillée (MovieDetail / TvDetail)

Affiche haute résolution avec backdrop, synopsis, genres, durée, date de sortie, casting cliquable, bande-annonce YouTube, plateformes de streaming, films similaires, notes et commentaires CLAP!.



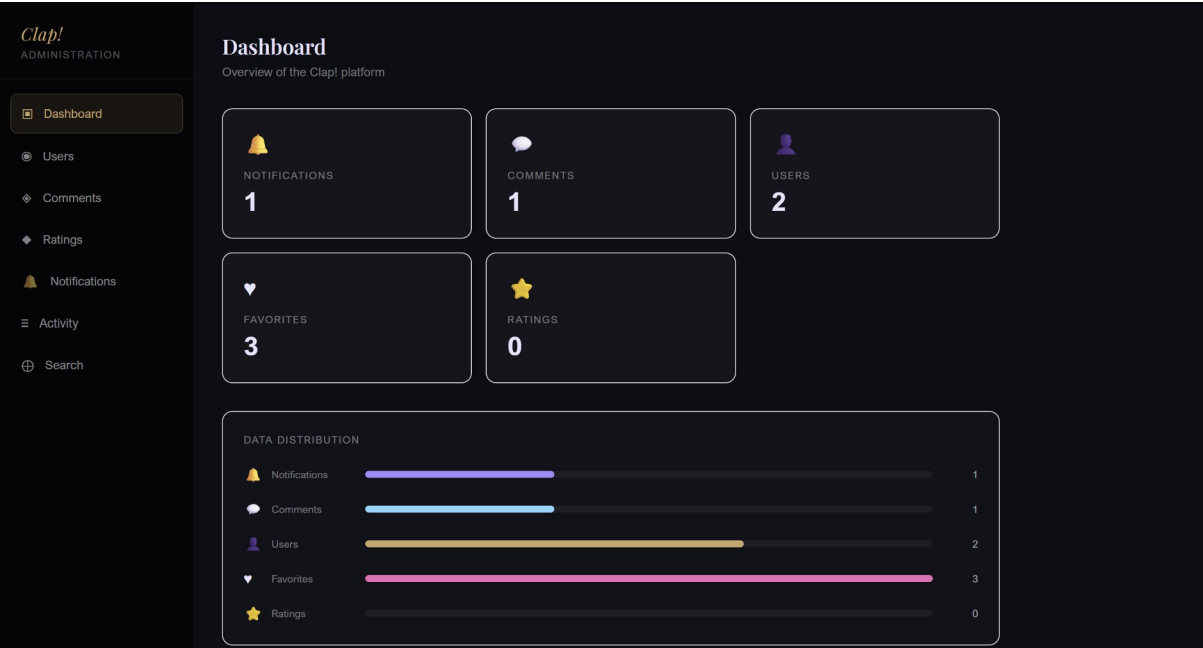
Profil utilisateur

Bannière personnalisée (backdrop TMDb), avatar, informations du compte, niveau XP, onglets Favoris / Listes / Historique, formulaire de modification du profil.



Interface d'administration

Back-office avec layout propre (AdminLayout sans Navbar globale), sidebar de navigation, tableau de bord, gestion des utilisateurs, modération, journaux d'audit, recherche unifiée.



09

DÉPLOIEMENT ET CONTENEURISATION

Architecture Docker

L'intégralité de la stack CLAP! est conteneurisée. Chaque service dispose de son propre Dockerfile multi-étapes : une étape de build Maven (image openjdk:17-jdk) produit le JAR, puis une étape runtime légère (openjdk:17-jre-slim) exécute l'application. Le frontend utilise une image Node pour le build React puis Nginx Alpine pour le runtime.

```
# Exemple Dockerfile multi-étapes — auth-service
FROM maven:3.9-openjdk-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package -DskipTests

FROM openjdk:17-jre-slim
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Organisation des conteneurs Docker Compose

Docker Compose orchestre les 9 conteneurs. L'ordre de démarrage est géré via `depends_on` et `condition: service_healthy`. J'ai configuré des health checks sur chaque service pour garantir que les dépendances sont prêtes avant le démarrage des services consommateurs.

Conteneur	Port exposé	Dépendances	Volume
mongodb	27017	—	mongo_data
consul	8500	—	—
db-service	8060	consul (healthy)	db_data
auth-service	8090	db-service, consul	—
movies-service	9000	db-service, consul, mongo	—
social-service	9010	db-service, consul	—
notifications-service	9020	db-service, consul	—
gateway	8080	tous services	—
frontend	5173	gateway (started)	—

Extrait docker-compose.yml

```

services:
  consul:
    image: hashicorp/consul
    command: agent -dev
    ports: ["8500:8500"]
    healthcheck:
      test: ["CMD", "consul", "members"]
      interval: 10s

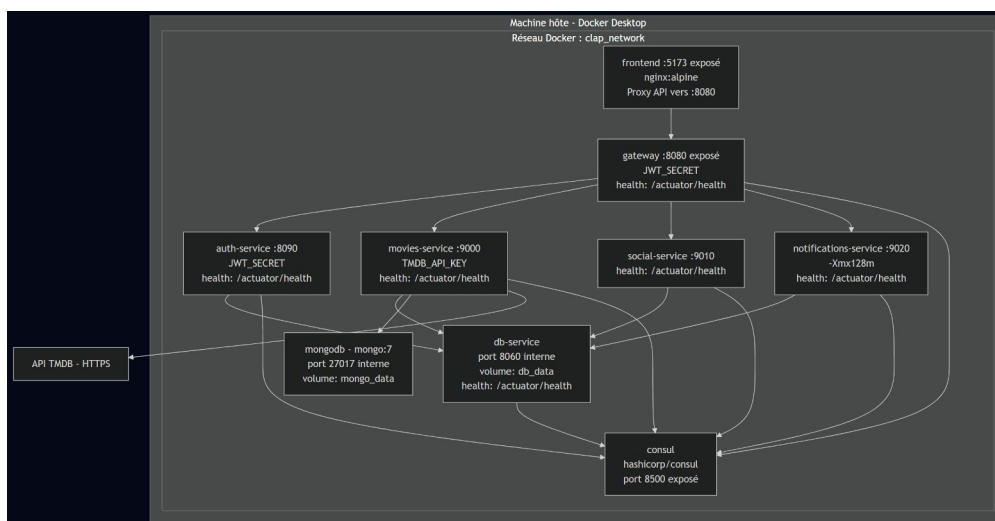
  db-service:
    build: ./Services/db-service
    ports: ["8060:8060"]
    volumes: ["db_data:/data"]
    depends_on:
      consul: { condition: service_healthy }
    environment:
      - SPRING_DATASOURCE_URL=jdbc:h2:file:/data/clap;MODE=MySQL

  gateway:
    build: ./gateway
    ports: ["8080:8080"]
    environment:
      - JWT_SECRET=${JWT_SECRET}
    depends_on:
      auth-service: { condition: service_healthy }
      movies-service: { condition: service_healthy }
      social-service: { condition: service_healthy }
      notifications-service: { condition: service_healthy }

```

Réseau interne et sécurité

Tous les conteneurs partagent un réseau Docker bridge dédié (réseau clap_network). Les services communiquent entre eux via leurs noms de conteneur (DNS interne Docker). Seuls trois ports sont exposés à l'extérieur : 5173 (frontend), 8080 (gateway) et 8500 (Consul UI). Le db-service, par exemple, n'est jamais accessible depuis l'extérieur.



Variables d'environnement

Les secrets ne sont jamais versionnés dans le dépôt Git. Un fichier `.env.example` documente les variables requises ; le fichier `.env` réel est ajouté au `.gitignore`. Les variables sensibles sont :

```
# .env.example
JWT_SECRET=<clé Base64 256 bits – générer avec: openssl rand -base64 32>
JWT_EXPIRATION=86400000    # 24 heures en ms
TMDB_API_KEY=<clé API TMDB – inscription gratuite sur themoviedb.org>
```

Démarrage de l'application

Prérequis : Docker Desktop 4.x+ et un fichier `.env` configuré.

```
# Cloner le dépôt
git clone https://github.com/antonin-tacchi/DWWM_Project_Title.git
cd DWWM_Project_Title

# Configurer les secrets
cp .env.example .env    # puis éditer .env

# Lancer l'intégralité de la stack
docker compose up -d --build

# Accès
# Frontend : http://localhost:5173
# Gateway : http://localhost:8080
# Consul UI : http://localhost:8500
# Swagger : http://localhost:8090/swagger-ui/index.html
```

10

PLAN DE TESTS ET JEU D'ESSAI

Stratégie de test

J'ai mis en place une stratégie de test à plusieurs niveaux, couvrant la logique métier critique, les endpoints REST et l'expérience utilisateur. Les tests backend sont écrits avec JUnit 5 et Mockito ; les tests frontend avec Vitest et `@testing-library/react`.

Tests unitaires backend

JwtUtil — Génération et validation des tokens

J'ai testé l'intégralité du cycle de vie du JWT : génération d'un token valide avec les claims attendus, extraction du `userId`, de l'email et du rôle, et détection des tokens expirés ou falsifiés. Ces tests garantissent que la couche de sécurité fonctionne correctement quelle que soit la configuration.

```
// JwtUtilTest.java
@Test
void shouldGenerateTokenWithCorrectClaims() {
    String token = jwtUtil.generateToken("user@test.fr", 42L, "user");
    assertEquals("user@test.fr", jwtUtil.extractEmail(token));
    assertEquals(42L, jwtUtil.extractUserId(token));
    assertEquals("user", jwtUtil.extractRole(token));
    assertFalse(jwtUtil.isTokenExpired(token));
}

@Test
void shouldRejectTamperedToken() {
    String tampered = jwtUtil.generateToken("x@x.fr", 1L, "admin") + "INVALID";
    assertThrows(JwtException.class, () -> jwtUtil.extractEmail(tampered));
}
```

Tests des contrôleurs

AuthController — Inscription et connexion

J'ai testé les scénarios nominaux (inscription réussie, connexion avec email, connexion avec username) et les cas d'erreur (email déjà existant, mot de passe incorrect, champs manquants). Les mocks Spring MockMvc simulent les requêtes HTTP sans démarrer de serveur.

CommentController et RatingController

J'ai testé la création, la modification et la suppression de commentaires et de notes, en vérifiant les contrôles d'autorisation (un utilisateur ne peut modifier que ses propres contenus) et la validation des entrées (score hors plage 1-10, commentaire vide).

Tests de sécurité

Scénario de test	Résultat attendu	Résultat obtenu
Accès /admin sans token	HTTP 401 Unauthorized	PASS
Accès /admin avec token user	HTTP 403 Forbidden	PASS
Accès /admin avec token admin	HTTP 200 OK	PASS
Token JWT expiré	HTTP 401 + déconnexion	PASS
Token JWT falsifié (signature invalide)	HTTP 401 Unauthorized	PASS
Doublon email à l'inscription	HTTP 409 Conflict	PASS
Note hors plage (0 ou 11)	HTTP 400 Bad Request	PASS
Injection SQL dans un champ texte	Rejeté par JPA/Hibernate	PASS

Tests fonctionnels

J'ai réalisé des tests de parcours complets sur l'interface utilisateur :

- Parcours inscription → connexion → recherche → ajout aux favoris → déconnexion
- Parcours notation et commentaire sur une fiche film
- Parcours création et gestion d'une liste personnalisée
- Parcours administration : gestion des utilisateurs et modération des commentaires

```
[INFO] Running JwtUtil ? génération et validation des tokens
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.081 s -- in JwtUtil ? génération
et validation des tokens
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.11:report (report) @ auth ---
[INFO] Analyzed bundle '' with 9 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:31 min
[INFO] Finished at: 2026-06-04T16:18:34+02:00
[INFO] -----
```

11

CONCLUSION

Bilan du projet

CLAP! est une application web complète, fonctionnelle et déployable en un seul comando (docker compose up). Elle démontre la maîtrise d'une stack technique moderne (React 18, Spring Boot 3.5, Docker, Consul) et l'application cohérente des bonnes pratiques du référentiel DWWM.

Objectifs atteints

- Application full-stack fonctionnelle avec backend microservices (5 services)
- Interface utilisateur responsive et animée (Three.js, Framer Motion, GSAP)
- Sécurité JWT complète : authentification stateless, RBAC, audit trail
- Intégration TMDB avec cache MongoDB et circuit breaker Resilience4j
- Conteneurisation complète (Docker Compose, health checks, volumes persistants)
- Internationalisation français / anglais (i18next)
- Panneau d'administration avec journaux d'audit
- Tests unitaires et fonctionnels couvrant les scénarios critiques

Difficultés rencontrées et enseignements

La principale difficulté a été la conception des frontières entre services. La question de centraliser ou distribuer la logique métier a nécessité plusieurs itérations avant d'aboutir à la solution du db-service centralisé. La gestion de l'ordre de démarrage Docker (health checks) et du cache TMDB (invalidation, clés déterministes) ont également demandé une attention particulière.

Ces défis m'ont appris à toujours implémenter les health checks dès la première itération, à documenter les décisions architecturales, et à tester les cas d'erreur avec autant de rigueur que les cas nominaux.

Perspectives d'évolution

- Migration vers Kubernetes (les health checks et la configuration sont déjà compatibles)
- Système de recommandation basé sur l'historique et les notes
- Refresh tokens pour éviter les déconnexions après 24h
- Application mobile React Native partageant les APIs backend
- Pipeline CI/CD avec GitHub Actions pour le déploiement continu
- Monitoring avec Prometheus et Grafana
- Migration de H2 vers PostgreSQL pour un usage en production

Ce projet représente l'aboutissement de ma formation DWWM : une application web professionnelle, sécurisée, maintenable et scalable, réalisée en totale autonomie. Il illustre non seulement les compétences techniques acquises, mais aussi la capacité à analyser un besoin, concevoir une solution architecturale adaptée et la mener à terme de façon rigoureuse.

GLOSSAIRE	
Terme	Définition
API Gateway	Point d'entrée unique d'une architecture microservices — gère routage, sécurité, CORS
BCrypt	Algorithme de hachage de mots de passe à facteur de coût adaptable
Circuit Breaker	Patron de conception protégeant contre les cascades de pannes dans les systèmes distribués
Consul	Outil de découverte de services et registre dynamique (HashiCorp)
Docker	Plateforme de conteneurisation permettant d'empaqueter une application et ses dépendances
Docker Compose	Outil d'orchestration multi-conteneurs via un fichier docker-compose.yml
Feign / OpenFeign	Bibliothèque Java pour créer des clients HTTP REST déclaratifs via des interfaces annotées
Flyway	Outil de migration de base de données basé sur des scripts SQL versionnés
H2	Base de données Java embarquée ou serveur, utilisée ici en mode fichier persistant MySQL
JPA / Hibernate	Spécification Java (JPA) et implémentation (Hibernate) pour le mapping objet-relationnel
JWT	JSON Web Token (RFC 7519) — jeton signé pour l'authentification stateless
MCD / MLD / MPD	Modèles Merise : Conceptuel, Logique et Physique de Données
Microservices	Style d'architecture décomposant une application en services autonomes spécialisés
MongoDB	Base de données NoSQL orientée documents — utilisée ici comme cache TMDB avec TTL
RBAC	Role-Based Access Control — contrôle d'accès basé sur les rôles utilisateurs
React	Bibliothèque JavaScript pour la création d'interfaces utilisateur composants
Resilience4j	Bibliothèque Java de résilience (circuit breaker, retry, rate limiter)
REST	Style d'architecture pour systèmes distribués basé sur les conventions HTTP
SPA	Single Page Application — application dont le contenu est chargé dynamiquement
Spring Boot	Framework Java simplifiant la création d'applications Spring autonomes
TMDB	The Movie Database — base de données collaborative de films et séries avec API publique
TTL	Time To Live — durée de vie d'une donnée en cache avant expiration automatique
Zustand	Bibliothèque de gestion d'état légère et sans boilerplate pour React

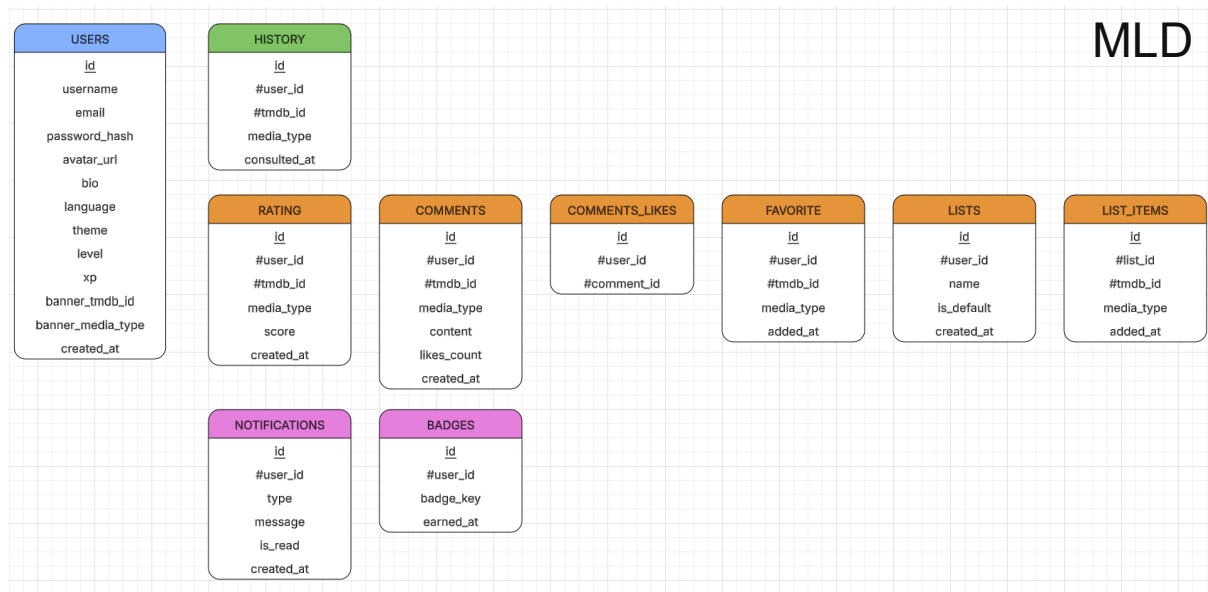
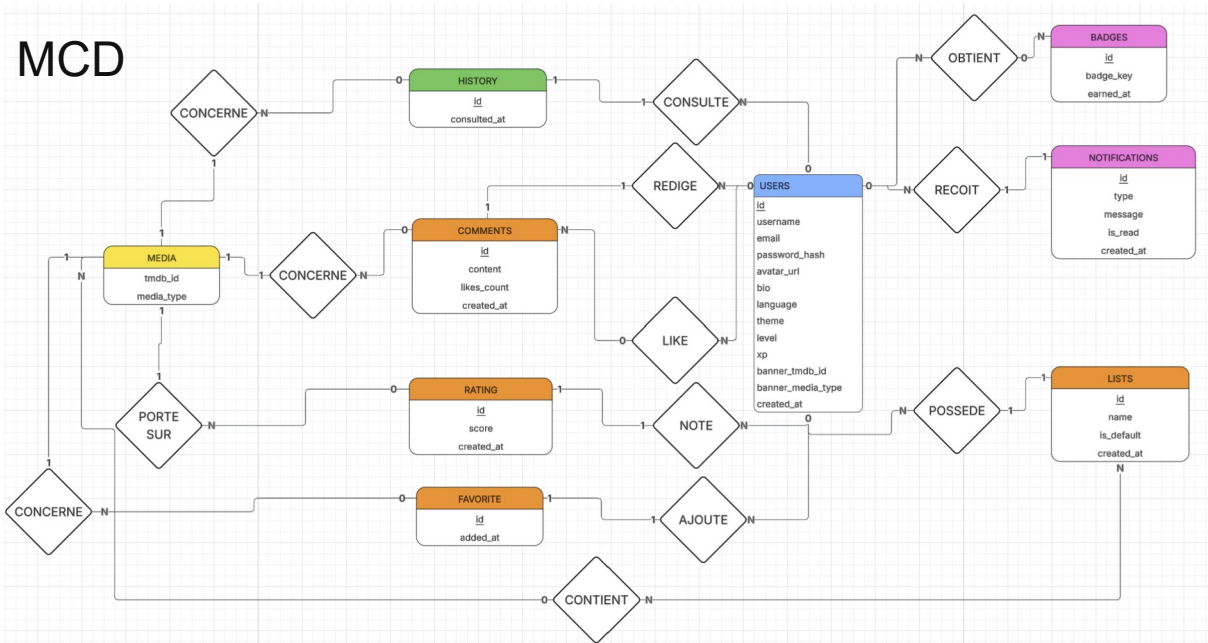
ANNEXES

Annexe A — Référence complète des endpoints

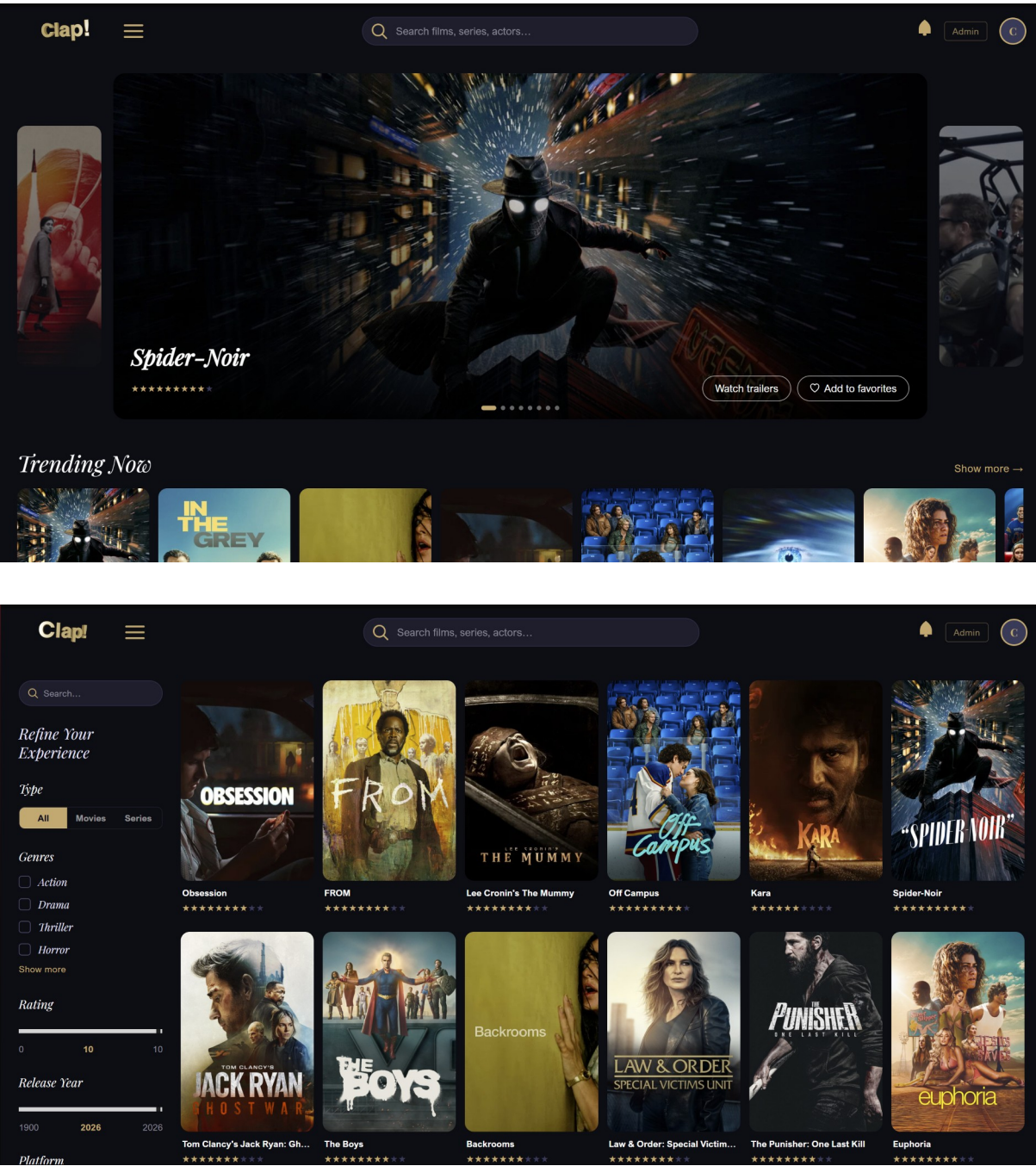
Méthode	Endpoint	Auth	Description
POST	/auth/register	Public	Inscription d'un nouvel utilisateur
POST	/auth/login	Public	Connexion (email ou username)
GET	/auth/me	Auth	Profil de l'utilisateur connecté
PUT	/auth/me	Auth	Mise à jour du profil
DELETE	/auth/me	Auth	Suppression du compte
PUT	/auth/me/password	Auth	Changement de mot de passe
GET	/admin/users	Admin	Liste de tous les utilisateurs
PATCH	/admin/users/{id}/role	Admin	Changement de rôle utilisateur
DELETE	/admin/users/{id}	Admin	Suppression d'un compte
GET	/admin/comments	Admin	Liste de tous les commentaires
DELETE	/admin/comments/{id}	Admin	Suppression d'un commentaire
GET	/admin/logs	Admin	Journaux d'audit administratifs
GET	/admin/dashboard	Admin	Statistiques du tableau de bord
GET	/movies/search	Public	Recherche de films et séries
GET	/movies/trending	Public	Contenus tendances
GET	/movies/popular	Public	Contenus populaires
GET	/movies/upcoming	Public	Prochaines sorties
GET	/movies/{id}	Public	Détails complets d'un contenu TMDB
GET	/movies/{id}/credits	Public	Casting et équipe technique
GET	/movies/{id}/providers	Public	Plateformes de streaming disponibles
GET	/actors/{id}	Public	Profil et filmographie d'un acteur
POST	/ratings	Auth	Attribuer une note (1-10)
GET	/ratings	Public	Notes d'un contenu
POST	/comments	Auth	Poster un commentaire
GET	/comments	Public	Commentaires d'un contenu
POST	/comments/{id}/like	Auth	Liker un commentaire

Métho de	Endpoint	Auth	Description
POST	/favorites	Auth	Ajouter aux favoris
GET	/favorites	Auth	Liste des favoris
POST	/lists	Auth	Créer une liste personnalisée
GET	/lists	Auth	Listes de l'utilisateur
GET	/notifications	Auth	Notifications de l'utilisateur
PATCH	/notifications/{id}/read	Auth	Marquer une notification comme lue

Annexe B — Schémas base de données



Annexe C — Captures d'écran de l'application



Clap!

Search films, series, actors...

Admin

81%
User Score

The Super Mario Galaxy Movie

Category: Family, Comedy, Adventure, Fantasy, Animation Release date: 04/01/2026 Streaming Platform: Only in Theater
Director: Michael Jelenic, Aaron Horvath, Pierre Leduc, Fabien Polack Duration: 1h 38m

▶ Watch Trailer

🔖 Save

Overview

Having thwarted Bowser's previous plot to marry Princess Peach, Mario and Luigi now face a fresh threat in Bowser Jr., who is determined to liberate his father from captivity and restore the family legacy. Alongside companions new and old, the brothers travel across the stars to stop the young heir's crusade.

Top Billed Cast

Clap!

Search films, series, actors...

Admin

Modify

Lvl 1HELLO

CapitainLicorne

0

Favorite Movies

1

Favorite Shows

0

Playlists

Favorite Genres

Crime

1

Drama

1

